

The Product Owner is Dead. Long Live Product Thinking.

“The king is dead. Long live the king.” — traditional proclamation of royal succession

If the team can catch the wrong shape, who catches the wrong ask?

Open source maintainers are drowning. I’m one of them.

The pattern is the same everywhere. Someone opens an issue asking for a quick way to verify a concept. A contributor picks it up, feeds it to an agent, and a pull request arrives. Thorough. Tested. Sometimes even clean. Sophisticated, extensible, production-ready. And wrong at the level of need. The maintainer now has to review the generated work, explain why it’s the wrong level of solution, and do it without discouraging the contributor from trying again.

Multiply that by twenty open pull requests.

Open source is just a distributed async team with an unstable roster and no onboarding. It always has been. Contributors come and go. The maintainers are the only constant. There’s no planning, no shared context that builds over time, no Product Owner holding the thread between handoffs.

I maintain OpenFeature, a feature flagging specification where behavioral correctness matters. A contributor once opened a large pull request, too large to review properly, so I asked them to split it into two for a better overview and an easier follow.

They came back with two pull requests, split by module. I’d meant by feature. The agent had also duplicated the shared logic between them instead of keeping it in one place. Technically compliant with my request. Structurally worse than what we started with.

They hadn’t been careless. The agent hadn’t malfunctioned. The instruction, split into two pull requests, was given without the context that would have made it meaningful. The agent filled the gap with the most obvious interpretation of my words, not my intent. What was missing

was *minimum viable context*¹: enough to rule out the obvious wrong reading. The agent did what it was told.

What followed was frustrating for both of us. A lot of back and forth. Tension that shouldn't have been necessary between two people who both wanted the right outcome. We got there eventually: two pull requests, one based on the other, a stacked diff that worked. But the path to it cost time, energy, and goodwill that a shared understanding of the problem upfront would have saved entirely.

Nobody validated the need or the approach before the agent ran.

What did I change after this? Honestly, not much, and that's worth saying directly. CONTRIBUTING.md gets ignored. Issue templates get skipped. In open source, a maintainer often can't have that conversation before a contribution arrives. The async distributed nature of it means the shared understanding has to be front-loaded somewhere else.

The best partial mitigation I've found is writing issues with more context: minimum viable context in the issue description itself, so the contributor, human or agent, has less to guess at. Not the full spec. Just enough to make the obvious wrong interpretation less obvious.

It doesn't prevent the problem. It makes it cheaper. And in open source, that's often the best available answer.

In product teams the same chain exists, though the Curse of Sisyphus chapter's product manager and this chapter's Product Owner (PO) are different roles that land in the same seat, not one person renamed. A stakeholder has a need. A PO interprets it. The team builds the interpretation. Somewhere in that chain the actual need gets distorted, not through malice, just through the normal compression that happens at every handoff. By the time it surfaces, something is already built.

The cost is rework. Delayed delivery. A PO who has to go back to a stakeholder and explain why what was built isn't what they meant. That conversation is never easy. And it keeps happening.

Product Owner, product manager, "Product": the titles vary and the industry has never agreed on the boundary. This chapter is about the seat, not the title: whoever sits between the stakeholders and the team, carrying both the product thinking and the organizational interface.

The PO exists, in part, to manage that chain. To be the translation layer between what stakeholders want and what teams build. To carry the context that doesn't fit in a ticket.

¹The name leans on minimum viable product; the discipline leans on the minimal reproducible example that open source and Stack Overflow have demanded for decades. Same instinct, aimed at understanding instead of bug reports.

To field the questions that surface mid-implementation when something doesn't quite add up.

It's a structural role for a structural problem and the role works until it becomes the structure's failure point.

The role doesn't have an off switch. Everything that can't be handled elsewhere routes through the same person. Stakeholder pressure from above. Requirement gaps from below. Product decisions that have to be made before anyone knows enough to make them confidently. POs I've known carried it well, until the structure broke them. Not because they were weak. Because it was set up to eventually break anyone.

Implementation got cheap. Discovering it was wrong didn't.

In the open source example, the maintainer pays that cost. In a product team, the PO pays it first, then the team pays it in rework, then the stakeholder pays it in delayed delivery. The misunderstanding moves through the system and someone absorbs it at every stage.

Collapsing the implementation phase surfaces misunderstandings faster. The agent runs, the output exists, the gap between what was needed and what was built becomes visible sooner. That's good if the spec was right. It's expensive if it wasn't.

A spec written before implementation starts and reviewed by the stakeholder catches the misunderstanding at the cheapest possible moment. The stakeholder reads it and says "that's not what I meant." Nothing has been built yet. The clarification costs a comment in a document, not two weeks of rework.

I do a lighter version of this without a spec at all. When a PO brings a ticket, I ask what the motivation behind it is: the need the ticket is standing in for. Often that need is smaller than the ticket. It could have been solved with something simpler, but the ticket has already been engineered into a full solution before anyone checked whether the need required one. The same wrong level as the open source contribution: a thorough answer to a question nobody validated. The question catches it one ticket at a time, before the build.

In the open source world that's a fast experiment before a sophisticated solution. In a product team it's a spec review before implementation starts, a review request on the spec instead of the code. The form is different. The principle is the same: validate the need before the thing gets built.

The traditional PO carries two things simultaneously. Product thinking: challenging intent, representing user needs, asking why before the team builds what. And stakeholder management:

navigating organizational pressure, translating business context into something a team can act on, carrying the political weight to push back when the request is wrong.

Those are different skills bundled into one role partly by necessity. Building the spec together changes the necessity.

When the team builds the spec together, when product thinking happens collectively, that first part starts to distribute. Senior engineers develop the habit of asking why before how. Tech leads challenge intent as part of the planning process, not as an afterthought. Product thinking becomes a broader team competency over time, rather than something concentrated in a single role.

That doesn't happen immediately. Some teams need a dedicated PO because that muscle isn't there yet. It becomes a development goal, not a permanent structure.

The second part doesn't distribute. Stakeholder navigation requires years of context that most engineers don't have and don't want to develop: the organizational interface, the political skill of managing what comes into the team from outside, the credibility to push back on an executive request. It's what survives as a dedicated role.

I did a version of this as a team captain. Requests came down from above, strange ones, impractical ones, and I made the call on them: some we didn't pursue, some we implemented differently than asked. As long as we honored the defined interface, the implementation was ours. That judgment, deciding what to absorb, what to push back on, what to reshape, is the part that doesn't distribute. It's not product thinking, and it's not something a team picks up in a planning session.

Something more like a Stakeholder Navigator, not a Product Owner in the agile sense. The person who gets the spec in front of the right people at the right moment, manages the review cycle, and protects the team from the noise while they do the thinking.

I know how that sounds. A rebranded PO. Partly yes. The difference is ownership. The old model has the PO owning product thinking and managing stakeholders. This one has the team owning product thinking and the navigator owning the organizational interface. That's a real split even if the job title looks similar from the outside.

And it's a more sustainable job, more defined, with an off switch.

The open source maintainer and the PO are dealing with the same structural problem from different angles.

The spec review is the mechanism that closes the gap between what's asked and what's needed.

A defined moment where the need gets validated before anyone builds anything: a checkpoint, not a planning ceremony. Where the stakeholder can say “that’s not what I meant” when it’s still cheap to hear it.

Birgitta Böckeler², analyzing the harness engineering practices that surround agent systems, names the limitation precisely: the harness can govern how code gets written without ever checking that it does what users actually need. Technical correctness and functional intent are different problems. The harness solves the first. The spec review solves the second.

The role evolution is a consequence of that. When the spec carries the shared understanding, when the team built it together and the stakeholder saw it before the build, the translation layer the PO was providing becomes less necessary. What remains is the person who managed the organizational interface well enough to get the spec in front of the right people in the first place.

That person still matters, maybe more than before, and they just don’t have to carry everything anymore.

Roles shift and responsibilities redistribute: the product thinking spreads into the team, and the organizational interface narrows to the one role that protects the room while the team does the thinking. All of it assumes the room exists. A place where the thinking actually happens, and a team that knows how to use it. That’s the thing nobody has built yet.

²Birgitta Böckeler, “Harness Engineering for Coding Agent Users,” in the “Exploring Gen AI” series (martinfowler.com/articles/exploring-gen-ai/harness-engineering.html). “Harness engineering” is a shared term — OpenAI uses it too (the Curse of Sisyphus chapter); the two arrived at it independently.